

18 | Vault（二）：阅后即焚功能实现解析

潘野 · 云原生基础架构实战课



你好，我是潘野。

上一讲，我们一起初步了解了 Vault 的架构和核心概念，并且还结合案例，实际练习了如何在 Kubernetes 上使用 Vault 管理和获取密钥。

这一次，我们要聊点更酷的东西，一起学习 Vault 的另一个神奇功能——租赁与回收机制。可能你并没有听说过这个功能，所以我们先从一个你更熟悉的场景聊起。

阅后即焚功能

想象一下，你在使用微信时，有没有使用过那个超级酷的“阅后即焚”小程序？就是你发送一个秘密消息给朋友，他们一看完，那消息就像被魔法师施了魔法一样消失不见了！

在企业安全管理中的妙用

你可能会好奇，这个功能与企业安全管理有何关联呢？

让我们先来对比一下传统的密钥管理方式——手动创建、分发、销毁密钥。听起来就很繁琐对吧？而且这样很容易出错。

这时候，“阅后即焚”功能就能大显身手了。它能自动管理密钥的整个生命周期，简化操作流程，大幅降低出错率，使得密钥管理变得既简单又安全。

我来举个例子，想象一下你和几个同事在管理一个分布式系统，需要在多个数据中心的服务器之间共享密码。一般来说，大家都是在内部聊天软件中直接发送。但是，这种使用传统方法传递密码的方式，很容易在内部的审计系统和日志系统中留下痕迹。一旦密码泄露，整个系统的安全都可能受到威胁。

但如果使用了类似“阅后即焚”的功能情况就不一样了。你把密码分享给一起协作的同事，设定只有指定同事能看，且只让他看一次。因为经过加密和权限设置，同事接收到密码之后，密码就会自动消失；如果未按时查看，密码也会消失，避免泄露。整个传输过程是加密的，并且是点对点的传递，且密码销毁及时，可以极大降低遭受攻击的风险。

在这个过程中，有两个特别重要的点：

查看后自动销毁：这确保了信息只被授权的人看到一次，之后就无迹可寻。

超时自动销毁：如果没在规定时间内查看，信息也会自动消失，又增加了一重保护。

这两点在 Vault 里头，就对应着**租赁与回收**这两个实用的功能。通过这种方式，Vault 不只是保护了密钥的安全，还能智能地管理密钥的生命周期，让一切变得既安全又便捷！

租赁与回收的例子

接下来我们结合例子，来看看这两个功能的具体实现。

假设你在一个云环境中管理应用，需要从 Vault 获取数据库的访问凭证。

1. 请求凭证：你的应用通过 API 向 Vault 请求访问特定数据库的凭证。

2. 接收凭证和租赁信息：Vault 处理这个请求，生成一个数据库用户名和密码，并创建一个租赁。假设租赁时长设置为 24 小时。
3. 使用凭证：应用使用这些凭证来访问数据库，进行数据读写操作。
4. 租赁过期：如果 24 小时后你没有请求续约，Vault 会自动让这些凭证失效，并从数据库中撤销这些凭证，确保它们不会被再次使用。

租赁的机制与方法

在这个流程中涉及几个关键步骤，包括请求密钥、管理租赁以及必要时续约或撤销租赁。我们来看看实际是怎么操作的。

1. 请求密钥

首先，你需要从 Vault 请求一个密钥或凭证。这通常是通过执行一个 API 调用或使用 Vault 的 CLI 命令来完成的。例如，如果你想从 Vault 中获取一个数据库的凭证，就可以通过后面这条命令完成。这条命令会根据 `my-role` 的配置生成一个数据库用户名和密码。

```
1 vault read database/creds/my-role
```

 复制代码

2. 接收租赁信息

执行上述命令后，Vault 不仅会返回所请求的凭证，还将返回与这些凭证相关的租赁信息。这包括：

租赁 ID (Lease ID)：这是一个唯一标识符，用于追踪和管理租赁。

租赁时长 (Lease Duration)：这表明了凭证的有效期限，通常以秒为单位。

续约策略：指示这个租赁是否可以续约以及如何续约。

例如，返回的信息可能看起来是这样的：

[📄 复制代码](#)

```
1 {  
2   "lease_id": "database/creds/my-role/1b4a7a00-f211-d3e9-3b8c-cea7b6f05a0c",  
3   "lease_duration": 3600,  
4   "renewable": true  
5 }
```

3. 租赁的续约

如果租赁是可续约的，并且你希望在租期到期之后，能够继续使用这些凭证，这时就需要请求续约租赁。

使用 Vault CLI，你可以执行如下命令来续约租赁。

[📄 复制代码](#)

```
1 vault lease renew [lease_id]
```

其中 [lease_id] 是你在租赁信息中获得的 Lease ID。你也可以指定续约的时长，前提是这不超过最大租赁时长的限制。

4. 租赁的撤销

如果你不再需要这些凭证，或者出于安全考虑需要立即使它们失效，就可以撤销租赁。使用 Vault CLI，执行如下命令将立即结束租赁，让相关的凭证失效。

[📄 复制代码](#)

```
1 vault lease revoke [lease_id]
```

自动回收的机制

一旦租赁到期，如果没有续约凭证，Vault 会自动执行回收操作，确保这些敏感信息不会被未授权访问或留存超出其必要的时间。

具体分成两个步骤。

1. 撤销密钥或凭证：相关的密钥或凭证将被自动从系统中删除，使其无法再被使用。
2. 记录日志：Vault 通常会记录相关的撤销操作，以便于审计和监控。

这个过程完全自动化，大大减少了出现人为错误的可能性。通过这种方式，即使凭证被泄露，它们也只有一个非常有限的时间窗口内有效，大大降低了安全风险。同时，自动化的租赁和回收机制也减轻了管理负担，使得操作更加高效和安全。

设置自动回收

自动回收通常是在设置 Vault 密钥引擎（Secret Engine）时配置的，例如在配置数据库密钥时，你可以指定租赁的默认时长和最大时长。后面就是一个设置数据库密钥的示例，首先建立一个密钥，允许访问该数据库连接的 Vault 角色为“db-role”。

 复制代码

```
1 vault write database/config/my-database \  
2     plugin_name=postgresql-database-plugin \  
3     allowed_roles="db-role" \  
4     connection_url="postgresql://{{username}}:{{password}}@localhost:5432/mydb?ss  
5     username="vaultuser" \  
6     password="vaultpass"
```

接着，你可以为特定角色配置租赁的时长，通过执行这个命令，Vault 将创建或更新一个名为“db-role”的数据库角色，该角色关联到之前配置的“my-database”数据库链接。当使用该角色请求动态数据库凭据时，Vault 将执行指定的创建语句来创建一个新的数据库用户，并将生成的凭据返回给请求者。

请注意，你需要将 `creation_statements` 中的 `"..."` 替换为实际的 SQL 语句，以根据你的数据库类型和所需的权限来创建用户。

 复制代码

```
1 vault write database/roles/db-role \  
2     db_name=my-database \  
3     creation_statements="CREATE USER {{username}} WITH PASSWORD '{{password}}';"
```

```
3 creation_statements="..." \
4 default_ttl="1h" \
5 max_ttl="24h"
```

在这个例子中，`default_ttl="1h"` 指定了动态生成的数据库凭据的默认生存时间（TTL）为 1 小时。这意味着如果在请求动态凭据时没有显式指定 TTL，则默认情况下凭据将在 1 小时后过期。

类似地，`max_ttl="24h"` 指定了动态生成的数据库凭据的最大生存时间（TTL）为 24 小时。这意味着请求动态凭据时指定的 TTL 不能超过 24 小时。一旦租赁期满，而且没有进行续约，Vault 将自动撤销该租赁。

监控自动回收

虽然自动回收是自动进行的，但为了确保回收正确可靠地运行，我们还需要监控这一过程，特别是在生产环境中。Vault 提供了日志功能，你可以通过查看日志来监控租赁的撤销情况。如果你配置了 Vault 的审计日志，撤销操作会被自动记录下来。

程序实现

前面我们学习了租赁与回收的机制和操作方法。回顾之前的例子，在实际工作中，我们其实不太可能直接使用 `vault` 命令来分享或获取密钥。一方面，操作过程中容易出错；另一方面，这种方式也不符合我们追求的自动化原则。

接下来，我们将通过一段 Go 语言代码，来说明如何用代码方式来使用 Vault 的租赁与回收功能。

首先，确保你已经安装了 Vault 的 Go 客户端库。如果还没有安装，可以通过下面的命令安装。

```
1 go get github.com/hashicorp/vault/api
```

 复制代码

或者在 go.mod 中引用 `github.com/hashicorp/vault/api` 包，这是由 Vault 官方提供的 Go 客户端。

然后，你可以创建一个简单的 Go 程序来实现这些功能。这段代码非常短，我来简单地解释下这段代码都做了什么。

首先是初始化，里面有两步。第一步是设置 Vault 服务器地址，程序是从环境变量 `VAULT_ADDR` 和 `VAULT_TOKEN` 中读取 Vault 的地址和访问令牌。你可以使用其他方式将 Vault 地址导入程序中。第二步是使用 Vault 的地址在程序内创建一个新的 Vault 客户端。

接下来是读取密钥，从 Vault 中读取一个密码，在这个例子中是 `secret/data/my-secret`。接着在终端中打印出获取的密钥数据和租赁信息。

获取到租赁信息之后，你可以延长租赁时间，也可以撤销租赁并清理资源。比如后面这段代码，就表示我们是在程序退出前使用了 `Revoke` 函数撤销了租赁，而且清理了资源。

 复制代码

```
1 package main
2
3 import (
4     "fmt"
5     "log"
6     "os"
7     "time"
8
9     "github.com/hashicorp/vault/api"
10 )
11
12 func main() {
13     // 设置 Vault 服务器的地址
14     vaultAddr := os.Getenv("VAULT_ADDR") // 例如: "http://127.0.0.1:8200"
15     if vaultAddr == "" {
16         log.Fatal("VAULT_ADDR environment variable is not set")
17     }
18
19     // 创建 Vault 客户端
20     config := &api.Config{
21         Address: vaultAddr,
22     }
23 }
```

```

24  client, err := api.NewClient(config)
25  if err != nil {
26      log.Fatalf("failed to create Vault client: %s", err)
27  }
28
29  // 设置 Vault token
30  vaultToken := os.Getenv("VAULT_TOKEN") // 从环境变量中获取
31  if vaultToken == "" {
32      log.Fatalf("VAULT_TOKEN environment variable is not set")
33  }
34  client.SetToken(vaultToken)
35
36  // 从 Vault 读取密钥
37  secret, err := client.Logical().Read("secret/data/my-secret")
38  if err != nil {
39      log.Fatalf("failed to read secret: %s", err)
40  }
41
42  // 输出获取的密钥
43  fmt.Printf("Secret data: %v\n", secret.Data["data"])
44
45  // 获取租赁信息
46  leaseID := secret.LeaseID
47  leaseDuration := secret.LeaseDuration
48
49  fmt.Printf("Lease ID: %s\n", leaseID)
50  fmt.Printf("Lease Duration: %d seconds\n", leaseDuration)
51
52  // 如果需要, 可以在程序结束前撤销租赁
53  defer func() {
54      log.Println("Revoking lease...")
55      if _, err := client.Sys().Revoke(leaseID); err != nil {
56          log.Fatalf("failed to revoke lease: %s", err)
57      }
58      log.Println("Lease revoked")
59  }()
60
61  // 模拟使用秘密一段时间
62  time.Sleep(10 * time.Second)
63 }

```

总结

今天, 我们主要学习了使用 Vault 进行密钥管理的高级功能——租赁与回收机制。

通过类比微信的“阅后即焚”功能，我们详细了解了 Vault 的租赁与回收功能如何工作，包括请求密钥、接收租赁信息、租赁的续约和撤销。这些功能使得密钥在使用后可以被自动回收，减少了密钥泄露的风险，并且减轻了管理负担。

企业安全管理中，密钥的租赁与回收常用于这些场景：

分享临时密码或一次性密码，例如用于登录网站或账户。

发送包含敏感信息的私人消息，例如财务信息或个人信息。

共享机密文件，例如企业销售数据等等

我们还通过一段 Go 语言代码示例，学习了如何在实际编程中使用 Vault 的 API 来实现密钥的获取、使用和撤销。代码中包括如何设置和使用 Vault 客户端、如何从 Vault 获取秘密信息以及如何在不再需要时撤销秘密的租赁，你可以课后自己动手练习一下。

思考题

不同类型的敏感数据（如数据库凭证、API 密钥等）对租赁时间的需求可能不同。想一想，我们应该如何优化租赁与回收策略呢？

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

AI智能总结

1. Vault的阅后即焚功能简化了密钥管理操作流程，提高了安全性。
2. 阅后即焚功能在企业安全管理中可以避免密码泄露，提高系统安全性。
3. Vault的租赁与回收机制智能地管理密钥的生命周期，确保信息只被授权的人查看一次，并在规定时间内自动销毁信息，提高安全性。
4. 租赁机制包括请求密钥、接收租赁信息、租赁的续约和撤销，有效管理密钥的使用和销毁。
5. 自动回收机制在租赁到期后自动撤销密钥或凭证，并记录日志，确保信息不会被未授权访问或留存超出必要的时间。
6. Vault提供了日志功能，可以通过查看日志来监控租赁的撤销情况，增加安全性。
7. Vault的租赁与回收机制自动化管理密钥的生命周期，减少人为错误的可能性，提高操作的高效性和安全性。

精选留言

由作者筛选后的优质留言将会公开显示，欢迎踊跃留言。